Verification Lab 3 & 4 - UVM

Mohammad Attari & William Marnfeldt

1 Introduction

In the previous lab, we learned some useful SystemVerilog features for building better testbenches. Although they can take us pretty far, you can imagine that the testing can get really complex for larger designs. Furthermore, having to remake testbenches every time we want to run variations is time consuming. Worry not, for there are more tools in our verification toolbox. So far we've been using the rusty tools found in the old tool shed. Now we are going to try using the industry standard power tools!

Universal Verification Methodology, or UVM for short, is a standardized methodology for verifying IC designs. It is commonly used in industry due to its reusability and modularity. Some of these benefits will be illustrated during the lab. If you are accustomed to Object Oriented Programming (or OOP, as the kids call it), you will recognize multiple concepts, namely inheritance and polymorphism. Some key takeaways from the following 2 labs are as follows:

- How to perform testing with UVM.
- How to structure a UVM testbench.
- How to minimize work with re-usability.

There will probably be tons of good questions and exactly zero bad ones during the lab[1]. I highly encourage you to ask them as soon as they pop up in your head. This can get pretty convoluted in the beginning, as there are quite many moving parts. Working with UVM typically includes two main tasks, which are test generation and infrastructure improvement, so the labs have been structured to reflect this. In the first session, you will get to learn how to create tests within a UVM environment, and in the second one you will focus on modifying the testbench itself. Before you get started, heres some good classic things that even the old guard get wrong every now and then.

- In the UVM-labs, run the test to completion. Use Run All.
- When QuestaSim asks you if you want to finish, select No. If you press yes, QuestaSim will close down.

[1] Don't ask me why.

2 The DUT & the TB

OK, just like last time, let's start by looking at what we have. This time around we have not just one, but two DUTs. Additionally, we have a separate directory with verification components and shell scripts to run each individual test. As you may notice by the length of the following list, there are quite a few files in this lab. This is in fact a result of the very standardized approach used in UVM, which involves some pain to set things up, in order to save time and reduce suffering later on.

Let's familiarize ourselves with the directory structure and our available files:

```
Lab2
├── data_to_fifo
│   ├── dut
│   │   └── data_to_fifo.sv
│   └── tb
│       ├── base_test.svh
│       ├── basic_test.svh
│       ├── scoreboard.svh
│       ├── tb_env.svh
│       ├── tb_pkg.sv
│       ├── tb_top.sv
│       └── top_config.svh
├── serial_to_parallel
│   ├── dut
│   │   └── serial_to_parallel.sv
│   └── tb
│       ├── base_test.svh
│       ├── basic_test.svh
│       ├── parity_test.svh
│       ├── scoreboard.svh
│       ├── tb_env.svh
│       ├── tb_pkg.sv
│       ├── tb_top.sv
│       └── top_config.svh
├── uvc - UVM Verification components (See Additional List).
├── vrun_data_to_fifo_basic_test.sh
├── vrun_serial_to_parallel_basic_test.sh
└── vrun_serial_to_parallel_parity_test.sh
```

There are three directories and three script files. The first two directories listed above contain the two separate designs and their testbench-related stuff. The `uvc` directory is explained in the next section, and the scripts are for your convenience to, simply put, run the tests with one click[2]. If you're getting nervous about the fact that there are two separate test benches, don't worry. We will only be working with with `serial_to_parallel`. The other one is there to use as a reference.

[2] OK, maybe more than one click, but who's counting?

2.1 UVC - Universal Verification Components

You think the previous list was long? Then brace yourselves for another one. Let's take a mid-list break and ask ourselves what a UVC is and why it felt it should get its own list. As with all fields of hardware design, you will find an overabundance of Three Letter Acronyms (TLA). For today, the most important one is probably UVC or the Universal Verification Components. The UVC is principally what we use in our test-benches to drive and monitor our DUT. Typically, we also try to abstract our signals away in order to make them more user-friendly. In other words, instead of fiddling with individual signals and throwing them haphazardly at the DUT directly during testing, we try to package these signals into *items* and play with those instead. This way, we create a potent test apparatus that allows us to scale up easily, as changing the stimuli now boils down to just modifying the sequence of items. For instance, if you are testing a memory module, you could create *write* and *read* sequence items, and create your test around these items instead of playing with memory signals directly.

OK, enough of a break[3], let's go back to our beloved long list.

```
uvc
├── clock_uvc
│   ├── clock_agent.svh
│   ├── clock_config.svh
│   ├── clock_driver.svh
│   └── clock_if.sv
├── data_fifo_uvc
│   ├── data_fifo_agent.svh
│   ├── data_fifo_config.svh
│   ├── data_fifo_driver.svh
│   ├── data_fifo_if.sv
│   ├── data_fifo_monitor.svh
│   ├── data_fifo_seq_item.svh
│   └── data_fifo_seq_seq.svh
├── parallel_data_uvc
│   ├── parallel_data_agent.svh
│   ├── parallel_data_config.svh
│   ├── parallel_data_driver.svh
│   ├── parallel_data_if.sv
│   ├── parallel_data_monitor.svh
│   ├── parallel_data_seq_item.svh
│   └── parallel_data_seq_seq.svh
├── reset_uvc
│   ├── reset_agent.svh
│   ├── reset_config.svh
│   ├── reset_driver.svh
│   ├── reset_if.sv
│   ├── reset_monitor.svh
│   ├── reset_seq_item.svh
│   └── reset_seq_seq.svh
└── serial_data_uvc
    ├── serial_data_agent.svh
    ├── serial_data_config.svh
    ├── serial_data_driver.svh
    ├── serial_data_if.sv
    ├── serial_data_monitor.svh
    ├── serial_data_seq_item.svh
    └── serial_data_seq_seq.svh
```

[3] Was it really a break? I want my money back.

Wow, this could almost fill a tome, but that's life[4]. It turns out that most UVC:s contain similarly named components. While it may look long and repetitive, it is actually quite useful. If we understand their main purpose, it makes all of this code really navigable and understandable. Let's briefly summarize what they do:

- **Agent** - A container class for a UVC agent.
- **Configuration** - The local configuration for said agent.
- **Driver** - The Driver decomposes a sequence items and controls what the interface should be sending to the DUT.
- **Interface** - The Interface is how we glue together our agent to the DUT.
- **Monitor** - This thing listens to signals from the DUT, and processes the output according to the design. Typically, this is sent to a scoreboard.
- **Sequence Item** - The internal packages used for signaling within an agent. This should contain all necessary information required to generate a transaction.
- **Sequence** - A series of sequence items that are put together to make a more realistic scenario. For example a write command followed by some payload.
- **Sequencer** - In this lab they don't have any class extensions and no separate file. Anyway, they execute a sequence, and break it down into sequence items, which are then sent to the driver.

All of the above-mentioned machinery forms a UVC agent. As you see, one is typically used for every individual interface. This is a good practice, as it reduces spaghetti code and allows us to operate the different interfaces independently. Let's try and stitch most of this stuff together in a more meaningful way. Figure 1 illustrates a typical setup of a UVM agent. Note that the UVC agent in Figure 1 contains both a means for writing (sequencer & driver) as well as a way of managing inputs. Not all UVC agents need to contain all of these components. Just imagine how silly it would be to make a monitor for a simple clock. Similarly, if your agent only reads, there is not necessarily a need for stimuli generation. To sum up, you may not find all of the components in every UVC.

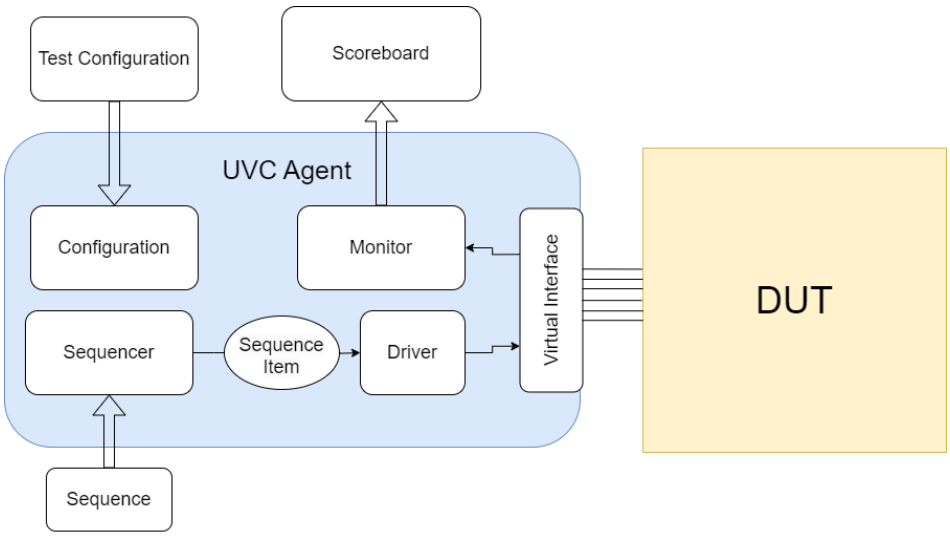


Figure 1: 10,000-foot view of how everything is interconnected within a UVM agent.

We're going to take a brief break[5] from UVM, just when it was getting interesting. Duty calls and we should really take a look at our DUT:s and the rest of the testbench environment.

[4] Sånt är livet. C'est la vie. So ist das Leben. Así es la vida.
[5] Oh boy!

2.2 The DUT:s

You may have noticed that there are two different DUT:s (in the `dut` subdirectories). The first one is a FIFO buffer and the second one is a serial to parallel converter. Take a look at their definitions and make yourself comfortable with the designs before you move on.

2.2.1 FIFO Buffer

As the name suggests our first DUT at hand is a First In First Out (FIFO) buffer. It can store up to 4 values at a time, with each value consisting of one byte. It will only store when valid is set high and there is no parity error, indicated by the `parity_error` signal. In these labs we will mostly use its testbench as a reference.

2.2.2 Serial to Parallel Converter

The serial to parallel converter simply converts 8 serial bits into an 8-bit parallel word. Furthermore, it can check for parity using the `parity_enable` signal. In such a case, it will expect an additional parity bit at the end for a total of 9 bits of input. If a parity error occurred on the input, this will be indicated by the `parity_error` signal.

2.3 The TB

In what follows I'm gonna do my absolute best to try and explain to you how the UVM flow works in a simple way. To get the most bang for your buck, check out the relevant files' contents as you make your way through this section.

Alright, navigate to the `serial_to_parallel/tb` directory. Just as in a traditional test-bench, it all starts in `tb_top`[6]. We start by introducing all of the "cables" needed. For us, this means declaring *logic* variables, followed by interfaces, which we will be using to connect everything to the UVC:s. Next, we instantiate the DUT and hook it up to the signals we created earlier, and then we initialize our configurations. Constructing the top configuration will in turn construct all the internal configurations of the UVCs as well. Finally, we are ready to start the base environment, with three magic lines of code[7]:

```
1    // Start UVM test_base environment
2    initial begin
3        run_test("basic_test");
4    end
```

This tiny little `run_test` does way more than one may think. In fact, it starts the UVM environment. In essence it works by executing four principal phases, namely:

1. Build Phase - Builds the components for the test.
2. Connect Phase - Connects all of the built components together.
3. Run Phase - Simulates everything!
4. Clean Phase - Gathers the results from the scoreboard, performs computations if needed, and checks that they look good.

In actuality, each of these phases contain smaller sub-phases, but for simplicity's sake, let's settle for these four principal phases.

[6] Some say it all started with the Big Bang, but thank you, I'm sticking to my tb_top.
[7] If you say four lines, then you're fired.

3 Using the Environment

Okey-dokey, enough explanatory stuff, let's actually do some lab work. Let's start by getting everything running. This time around, we won't need to create a project using the GUI. Instead we will be using a few small scripts. How we do this depends a bit on if we are running QuestaSim on a school computer or via Apps Anywhere. Therefore I'll explain both.

3.1 Good 'ol Linux (Blåtunga/Rödnäbba)

If you're running on a school Linux machine, move the lab files into a suitable working directory. Open a terminal and navigate your way to your working directory. From there, either source to QuestaSim immediately, or alternatively you can try:

```
source sourceme
```

Now make sure everything works by calling the shell script below:

```
./vrun_serial_to_parallel_basic_test.sh
```

If everything works as intended, QuestaSim should launch with the proper settings. Remember that if things are acting up, read the logs. If it doesn't work and you get a "permission denied" message, try using the `chmod` command. This sets the read and write access on a file. It would look like this:

```
chmod 666 vrun_serial_to_parallel_basic_test.sh
```

3.2 Cool new Apps Anywhere

If you're running from Apps Anywhere, start your client as usual. It is recommended moving the lab files to somewhere that is easy to access, because you will have to navigate to them via the terminal window in QuestaSim. As with most terminals, the keyword is `cd`. I recommend copying your path from the windows explorer and pasting it into the QuestaSim terminal. You will need to do a slight modification. Change the backslashes in the the path to forward slashes. Figures summarize what you will need to do.

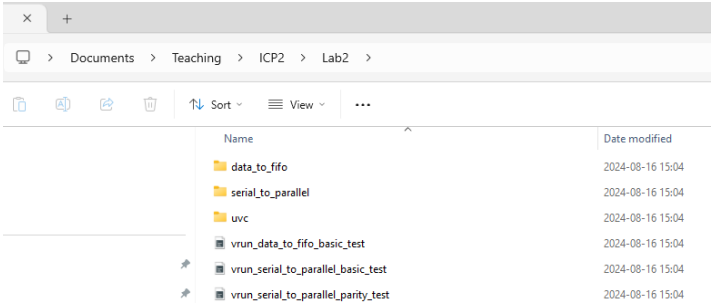


Figure 2: Navigate to your working directory.

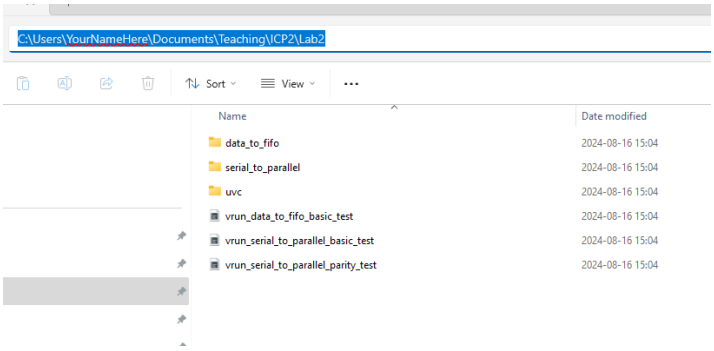


Figure 3: Copy your path!

```
QuestaSim> cd C:/Users/YourNameHere/Documents/Teaching/ICP2/Lab2
```

Figure 4: Modify the backslashes and add cd at the start.

Now you can call the shell scripts by typing:

```
source vrun_serial_to_parallel_basic_test.sh

```

4 UVM Session 1 - The agent

Pitter Patter! Let's get at'er![8] Although UVM is praised for reusability, every once in a while we will have to construct out agents. I have both good news and bad news for you:
The good news, this is probably the trickiest part of UVM.
The bad news, this is probably the trickiest part of UVM.
Get ready to traverse a few more files in this lab.

4.1 Our key components

In this lab we will be working on building up a **serial data UVC**. Building an agent is actually quite simple. All you gotta do is set up an **agent**, a **config**, a **driver**, a **monitor** a **sequence item**, and a **sequence**. Today you will be implementing parts of said classes. Due to the amount of boilerplate code, you will be provided with some skeleton code. We will mainly focus on getting things up and running.

4.2 The target interface

Typically when setting up a UVM testbench, we like to build agents on an interface basis. This is a good practice because multiple signals are commonly used in combination across a single interface. A common example is the AXI-interface, where the ready/valid signaling makes sense to have in the same driver in order to follow protocol. Let's take a look at the interface we need to drive in this lab. It can be found in `serial_data_if.sv`. Note that it is a very simple interface, with only three signals. These are:

- `start_bit` – A start bit to indicate that the current bit is the start of a word.
- `serial_data` – Here we actually send our data, one bit at a time.
- `parity_enable` – If this bit is enabled, we should include an additional 9th parity bit for error detection.

4.3 Driving our interface

Before we write a single line of code, we should consider what our serial data UVC should be doing. Since we are working with an 8-bit serial to parallel to converter, it is reasonable to build our agent as a little box that can generate output sequences of 8 serial samples. Let's start simple and gradually increase. For now we will ignore stimulating the parity. We will start by introducing glitch free stimuli every time a sequence item is sent to the driver. The **driver** needs to do a few things.

- In the run phase of the Driver, wait for a sequence item to appear. When it has appeared, take the 8-bit word `serial_data` in the sequence item.
- Serialize the word by writing one bit on the interface every clock cycle. The word should be written from right to left.
- When the first data-bit is being written, the `start_bit` should also be set to high. This can then be set low in the next cycle.
- When the word has been finished, send a response to the sequence, freeing up for the next transaction to begin.

Task 1.1:
Build a simple run phase for the `serial_data_driver`.
Verify that it works as intended by comparing output logs to the waveforms. **Hint: Remember, you can use for-loop and other non-synthesizable language structures in your testbench infrastructure. Work smarter, not harder.**

[8] Bonus points if you know which show that's from.

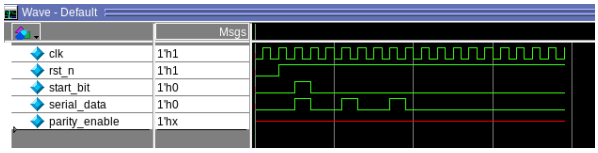


Figure 5: After task one you should be able to see something like this. In this example the word was 01001001.

4.3.1 Misalignment: A bug as a feature.

Ok, so let's do something counterintuitive. Let's implement a bug as a feature. As we need a test-bench capable of finding bugs, let's expand the possible stimuli we can generate. Let us introduce two new "features". If you have been looking in `serial_data_seq_item.sv` you may already know what we're going to add. The variables there, `start_bit_delay` and `start_bit_length` should allow us to delay the start bit and hold it high for a larger time, causing some unexpected behavior. Your next step is to set up the driver to be able to delay outputting the `start_bit` with a delay of `start_bit_delay` cycles. It should then be held high for `start_bit_length` number of cycles.

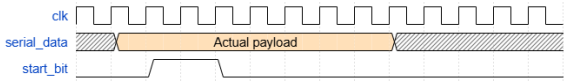


Figure 6: The start bit is delayed by one clock cycle and has a length of 2. For the sharp-eyed viewers out there, this is an invalid transaction. Yes, we actually want to generate this kind of nonsense, but in a controlled fashion.

Task 1.2:
Improve on the `serial_data_driver` by implementing start bit delay and start bit length.
Hint: Try using a fork join structure for an easier time. Testing these delays can be a bit tricky. However, we can easily control them. If you wish test a specific use case, take a look in the `serial_data_seq`.

4.4 On the parity bit.

Parity bits are commonly used in serial communication as cheap way of detecting bit errors. You may perhaps recognize it from using UART! In this lab we will look at implementing support in one our test benches for generating a parity bit. Although the task may seem simple, it provides ample opportunity to navigate different UVM components. Because this can be somewhat tricky the first time, we will only implement one additional signal and only in one test bench. In order to verify this feature, we will modify the appropriate files so as to be able to generate said stimuli. So let's get started! Before we start tweaking any code, we should stop and meditate on the matter. What are we actually going to implement? What is a parity bit? A parity bit is a simple mechanism for error detection in a transmission. Simply put, you count the number of 1:s in a sequence. If it is an odd number of ones, the parity bit is set to zero and vice versa. When the recipient then receives the message, they just perform a bit-wise XOR between all the bits, including the additional parity bit. If the result returns zero all is well. However, if the result becomes a one, something probably went wrong during the transmission. Figure 7 illustrates a transaction gone wrong with error detection.

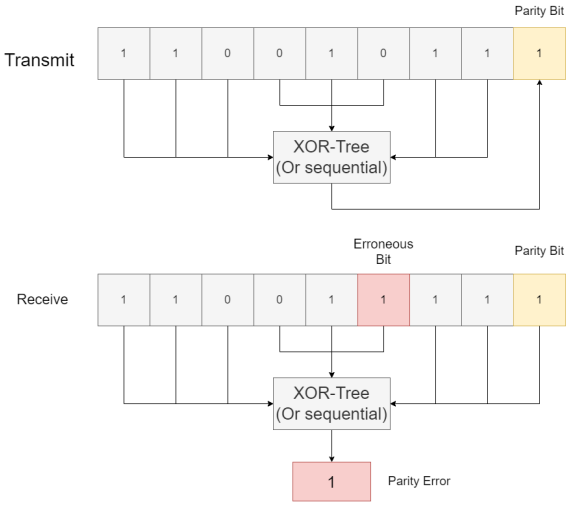


Figure 7: The parity bit is produced on the transmit side.

4.5 What to add and where.

Unlike the previous signals that we have been driving, the parity signal is mysteriously missing from most of the agent. Looking back at figure 1, we can get a feeling for where we will need to work today. Since we are looking at adding stimuli it ought to be related to the agent that drives the signal. However, the signal will need to be produced as well, and we will want to be able to monitor it, as well as be able to control it. Figure 8 provides an overview of this dependency chain.

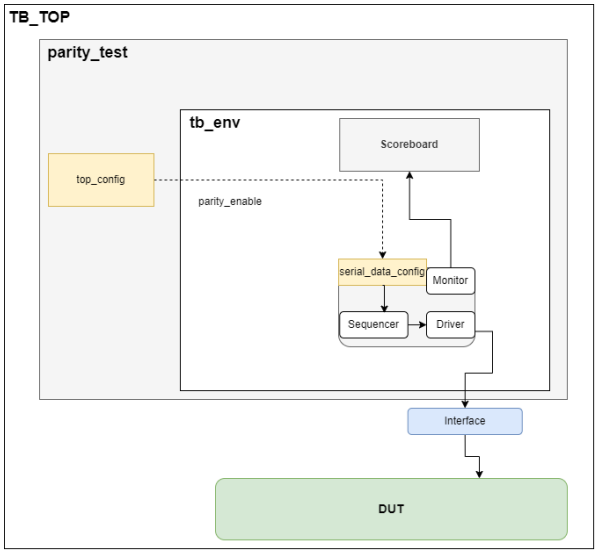


Figure 8: Adding our modification will require modifications in several places.

Here is a summary of how everything will work: We will create a test based off of the `base_test` we created last time. In principal it will function the same with one slight change. In the test, we will enable a setting in the `top_config`: `parity_enable`. The `top_config` will in turn change a local `parity_enable` flag in the `serial_data` UVC. After having propagated the test setting down to the actual UVC, we will have to modify the internal components of the UVC so that it generates an additional bit when the flag is set. In summary we are going to have to take a look at the following files:

- `top_config` - Adjust the settings to include a knob for enabling parity testing.
- `serial_data_config` Add a local knob to indicate if parity testing is used or not.
- `serial_data_driver` - If parity testing is enabled, set the `parity_enable` signal on the DUT high, and generate the 9th parity bit. Depending on if the sequence
- `serial_data_seq_item` - Include a randomizable parity error signal. This will be used in the driver to determine if the party bit is correct or not.
- `serial_data_monitor` - Not only do we want to generate stimuli, but also monitor it when we send it.

4.6 Implementing a parity bit.

In order to be able to test our progress, we will try and work our way from the bottom up. Let's start with something simple. Let's try and set the `parity_enable` signal high from the driver. Make sure to try out your changes frequently, to make sure it runs!

Task 2.1:
Modify the `serial_data_driver` so that it outputs a high `parity_enable` signal.
Hint: Look at how the other signals are referenced.

Good job! Your very first signal from scratch in UVM. Although this is a very cool signal and must be very special to you now, we're not done yet. Naturally, we don't always want it to be on. In fact, we should make it so that it is configurable. In order to do that we're gonna need to modify both of our configurations. This is where it gets a bit tricky. We now want to control the signal via a setting from our global configuration. In order to do this we gotta figure out how all of the references are set up. Fortunately i made a drawing...

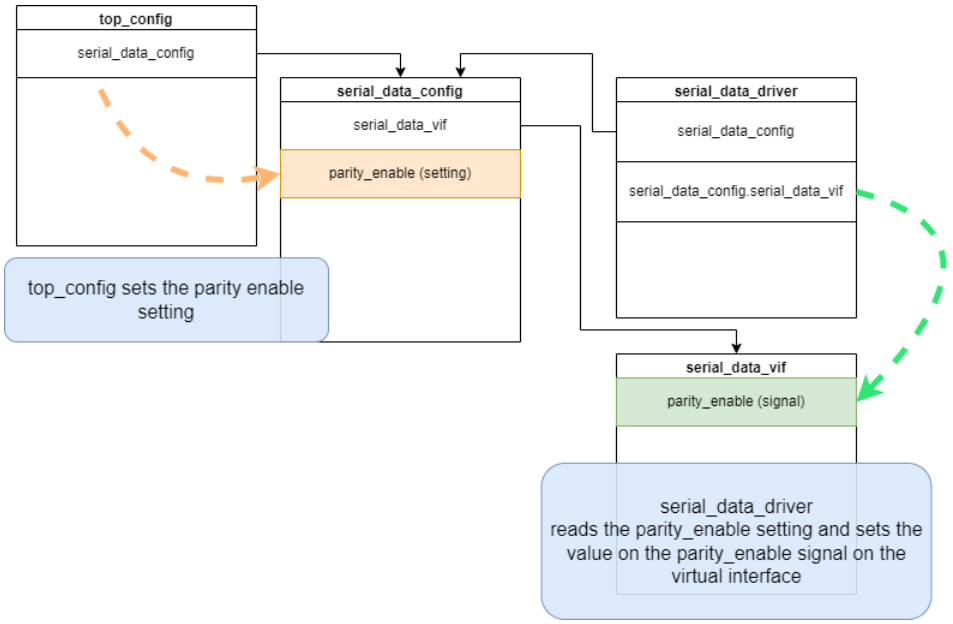


Figure 9: The top configuration should be able to set the parity_enable setting. The driver will then read that setting and set the parity enable_signal correspondingly.

Task 2.2:
Introduce a parity enable bit in the `serial_data_config`. Modify the `top_config` so that it initializes the value of `parity_enable` in its instance of `serial_data_config` to 0.
Finally, set the `parity_enable` signal on the virtual interface from the `serial_data_driver` to the value of the corresponding bit in `serial_data_config`.
Hint: Look at the image I made to try and explain it a bit better.

Nice! Now we should be able to control the output of `parity_enable` all the way from the `top_config`. We can in turn reach this from a test. In other words, we can enable create specific tests that run with parity or without. With this in place we should be able to run the parity test. It will still be quite lackluster, as we are only for the whole test setting a bit high or low. If we want to liven things up[9], we're going to have to keep building. Our next step should be actually generating a parity bit after we have sent the rest of our 8 bits. We will need to calculate the parity of the previous bits `serial_data` and send that as our parity bit. We won't implement parity errors quite yet.

[9] We do.

Task 2.3:
Implement a conditional 9th parity bit. This should only be generated if `parity_enable` is set high.
Hint: Look up what $countones() does. Can this be of any help?[a]

[a] I mean, obviously. I'm putting effort into this manual. Why would I include hints if they weren't useful?

This looked easy, but probably took a bit of testing. Now we can generate correct parity bits, but naturally we want to generate incorrect parity bits as well. Because we also wish to be able to keep track of that, we may want to adjust our sequence items. That way we can eventually register what happened from the monitor. Your next task is simple: Introduce a random `parity_error` bit. It should be included in the sequence item. If it is high, we should flip the `parity_bit`.

Task 2.4:
Implement a `parity_error` bit in the sequence item. If this bit is high, flip the 9th parity bit.

Now that everything should be in working order. Let's do some final polish. We want to be able to keep track of which sequences had faulty parity bits. We're gonna need to uncomment some code in the monitor.

Task 2.5:
Uncomment the code in the monitor. Should the test be working now?

At this point, we dont really know how to modify the tests, but try hardcoding the parity bits in the sequence to make sure that the parity bit works for both errors and without. In the next part, we will learn about working with our agent from a top down perspective.

5 UVM Session 2 - Same testbench, multiple tests

And at last, we can start running the testbench more usefully. Let's learn how to create tests using our agent that we constructed earlier.

5.1 Figuring out the basics!

Now that our agent is set up, we may as well try running the simulation!

Task 1.1a:
Run the simulation and take a look at the transcript. What do you see?

Currently, the simulation is very lackluster. If you, in addition to looking at the transcript window, view the waveforms, you will find that the input stimuli is very lackluster. Essentially it's a single reset signal, a clock and a small sequence. How about if we run the other testbench?

Task 1.1b:
Run the simulation for the other testbench. It is called `data_to_fifo_basic_test`. Take a look at the transcript. What do you see?

Hey! No fair! Why couldn't we get the one that has all the bells and whistles. Who removed all the fun stuff from the testbench we're working with? `:(` Let's try and figure out how all of this works. Because it's rather empty, we don't have much to go on, but as mentioned before, we do have a reset signal and a single sequence. Try and find where the reset signal is set in our UVM test.

Task 1.2:
Examine the `basic_test.svh` file. In which phase should the reset signal be created in?

Hmm... that's strange, the file is almost empty. There seems to be no mention of any reset signals. However, there are a few terms that may be familiar to you if you have worked with object oriented programming. Here are 2 examples:

- `class basic_test extends base_test;` - This is an inherited class.
- `super.run_phase(phase);` - The run phase calls the run phase of its parent.

So there seems to a be another run phase involved, and we now know that our `basic_test` is built off of `base_test`. Let's investigate that file.

Task 1.3:
Examine the `base_test.svh` file. Take a look at the run phase.

And Voilá! There it is! Our missing reset signal can be seen being generated here in the run phase:

```
62          // Reset DUT before start
63          reset = reset_seq::type_id::create("reset");
64          if (!(reset.randomize() with {
65              delay == 0;
66              length == 2;
67          }))`uvm_fatal(get_name(), "Failed to randomize reset")
68          reset.start(m_tb_env.m_reset_agent.m_sequencer);
69
```

This looks like a good entry point into improving upon our test. You will probably recognize some of the syntax as plain SystemVerilog as well. What is new is the creation of the `reset` sequence as well as `.start` method. With these lines of code we are simply creating a new sequence, with a given set of constraints and finally telling them to start running. A pretty comfortable way of writing stimuli.

5.2 Adding Serial Stimuli.

Alright, now that we have a basic understanding of how we can control the environment. As we are currently missing meaningful serial input stimuli, we will implement that. You have probably noticed that only one sequence is generated. As we saw in the previous labs, we typically want some repetition. Let's modify the test so that it generates new transactions `no_of_data_loop` times.

Task 2:
Modify the test so that it repeatedly generates serial sequences. Verify this by running the simulation.

5.3 Adding Mid Simulation Resets

Alright, this is starting to look really good. What would make it even better is if we had some mid-sim resets.This is commonly done in testing, as it is a good way to ensure that resets clear what they ought to clear and that the DUT can reset gracefully. Here we want to be a bit more random with when they occur, and we want them to be able to occur at the same time as serial data is sent. You may have been wondering what the strange `fork` and `join` keywords mean. Essentially, you spawn additional threads that the simulator can run in parallel, allowing multiple things to happen at once, even within a sequentially run task.

Task 3:
Modify the test so that it performs `no_of_resets number` of mid sim resets. Run your simulation and make sure it works!

If everything is working as it should, you should be seeing nice green coverage for everything but parity errors. It should look like in figure 10.

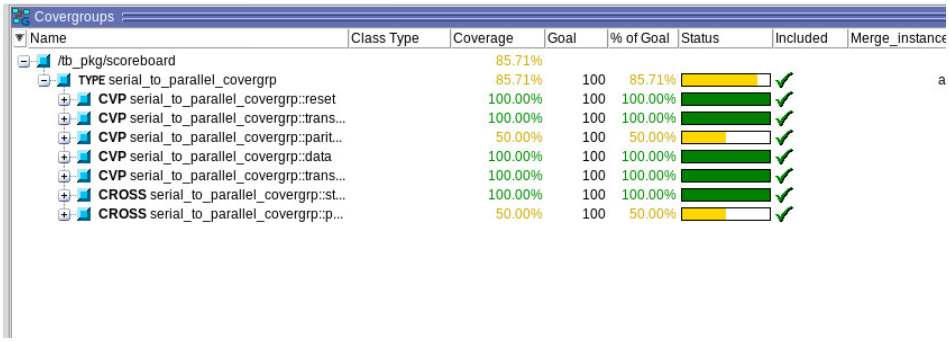


Figure 10: Your FCov should look like this. Don't worry about the parity errors. Yet...

Excellent, now we have something that is quite useful. As you have seen with good infrastructure, we can get some quite advanced testing going relatively quickly. Instead of writing long and clunky sequences of bits, we use random random sequences to quickly generate them. Our tests become simple invocations of various interfaces.

5.4 Finishing up

Ok, so let's finish up and see if we can full coverage in our scoreboard. Last time we implemented a parity bit. Make sure that it get randomized as well as parity errors. Fix everything up and get ready to show your work!

Task 4:
Use your new knowledge to achieve full coverage. Present your results to a TA when you are ready.

6 Feedback

Here comes the meta-textual part. These labs were the first labs I wrote (William) and I would love to hear what could be improved upon. Discussing UVM in such a short time-span[10] is challenging, requiring judgement calls about what is important and what isn't. I hope that the tasks felt useful yet challenging enough. Finding that sweet spot between hand holding and absolutely impossible is not as easy as one may assume. Anyway, if you have the spare time, please tell me what was good, what was bad and what was ugly.

[10] yes, believe it or not 8 hours total counts as short